

EE 3EY4

Lab 7

Ryan Li – li2323 – 400451015

Sungkyun Cho – chos45 - 400464608

March 17, 2026

Activity 1:

Equation (9) represents the second-order closed-loop differential equation derived from the vehicle kinematics. By applying the Laplace transform to (9) under zero initial conditions, we obtain the closed-loop transfer function. For a constant reference input of magnitude A, the steady-state error is determined by applying the Final Value Theorem to the error signal.

$$\ddot{d}_{lr} + k_d \dot{d}_{lr} + k_p d_{lr} = k_p d_{lr}^{des} \quad (9)$$

$$E(s) = D_{lr}^{des}(s) - D_{lr}(s)$$

$$H_u(s) = \frac{D_{lr}(s)}{D_{lr}^{des}(s)} = \frac{k_p}{s^2 + k_d s + k_p}$$

$$E(s) = D_{lr}^{des}(s) \left(1 - \frac{k_p}{s^2 + k_d s + k_p} \right)$$

$$\frac{A}{s} = \mathcal{L} \{ A u(t) \}$$

$$E(s) = \frac{A}{s} \left(1 - \frac{k_p}{s^2 + k_d s + k_p} \right)$$

$$e_{ss} = \lim_{s \rightarrow 0} s E(s) = \lim_{s \rightarrow 0} A \left(1 - \frac{k_p}{s^2 + k_d s + k_p} \right)$$

$$\therefore e_{ss} = A \left(1 - \frac{k_p}{0+0+k_p} \right) = 0$$

The calculations above demonstrate that the steady-state tracking error is zero for the given input. This result is consistent with control theory; given the assumption of unity feedback, the open-loop vehicle dynamics, derived from the closed-loop transfer function, characterize a Type I system, which inherently eliminates steady-state error for step inputs.

$$H_u(s) = \frac{k_p}{s^2 + k_d s + k_p} = \frac{G(s)}{1 + G(s)}$$

$$G(s) = \frac{H_u(s)}{1 - H_u(s)} = \frac{\frac{k_p}{s^2 + k_d s + k_p}}{1 - \frac{k_p}{s^2 + k_d s + k_p}} = \frac{\frac{k_p}{\cancel{s^2 + k_d s + k_p}}}{\frac{s^2 + k_d s}{\cancel{s^2 + k_d s + k_p}}}$$

$$G(s) = \frac{k_p}{s(s + k_d)}$$

$\therefore$ Type I system
 $e_{ss, \text{step}} = 0$

Increasing the proportional gain, k_p , reduces both the damping ratio and the rise time, resulting in a faster but more underdamped response. In terms of vehicle dynamics, this translates to a faster steering correction, but at the cost of more overshoot and transient oscillations. Vice versa, increasing the derivative gain, k_v , increases the damping ratio and rise time while reducing the settling time. This yields a more heavily damped response that, while slightly slower to initiate, reaches steady state more rapidly. For the vehicle, this results in a more gradual steering maneuver with minimal overshoot and faster convergence to the center path. As this is a Type I system, neither parameter affects the steady-state error for a step input. Proper calibration involves balancing these gains to achieve a response that is simultaneously responsive, stable, and quick to settle.

Activity 2:

$$v_s^c = v_s^d [1 - \exp^{-\max(d_{ob} - d_{stop}, 0)/d_\tau}] \quad (20)$$

The velocity command, v_s^c , is computed using Equation (20) based on the user-defined nominal velocity, v_s^d . The system's behavior is governed by the relationship between the obstacle distance (d_{ob}) and the minimum safety distance (d_{stop}). When no obstacles are present ($d_{ob} > d_{stop}$), the exponential term remains negligible, allowing the vehicle to maintain its nominal cruising speed.

As an obstacle is detected and the distance closes, the exponential term increases, causing v_s^c to attenuate. The rate of this deceleration is governed by the design parameter d_τ ; smaller values result in aggressive braking, while larger values provide a smoother, more gradual approach. If an obstacle enters the safety buffer ($d_{ob} \leq d_{stop}$), the command drops to zero, ensuring the vehicle maintains the required clearance. This exponential strategy ensures a smooth transient response during deceleration, though the system is designed to immediately stop if an obstacle suddenly appears within the d_{stop} threshold.

Activity 4:

During the simulation, the vehicle successfully maintains any speed from 1 m/s to 2 m/s. Upon start up, initial steering oscillations occur as the controller drives the vehicle down the hallway. These oscillations decrease quickly as the vehicle continues to drive straight down the hallway, and the system achieves stable tracking. However, geometric irregularities such as open doorway or people walking in the halls induce brief steering deviations. The system then promptly fixes itself once the wall profile stabilizes.

Navigation through corners introduces more significant challenges. The LiDAR sensor detects a sudden, large differential between the distances d_l and d_r , triggering an aggressive steering command. If the vehicle turns too sharply, it can lead to larger oscillations or even collisions, which we observed a few times during testing. As the vehicle enters the turn and acquires the new wall, the error decreases, leading to counter-steering and subsequent oscillations. This behavior still happens even when adjusting the k_p and k_d gains, as the step-like change in wall

geometry prevents a perfectly critically damped response. A primary limitation remains the system's "blind spot" during sharp turns. If a wall or obstacle appears suddenly after a corner, the vehicle may lack the necessary look-ahead distance to decelerate, emphasizing the algorithm's weakness to non-static or unexpected obstacles.

The k_p and k_d parameters significantly influence the vehicle's dynamic behavior. The proportional gain, k_p , drives the system toward the desired trajectory by responding to the current position error. While a high k_p improves responsiveness, excessive values can lead to instability and chronic oscillations. Vice versa, the derivative gain, k_d , introduces damping by opposing rapid changes in the error signal. While k_d is essential for suppressing oscillations, an over-damped system (high k_d) may react too slowly, potentially causing the vehicle to miss sharper turns. Experimental tuning showed that making k_p too high made the steering overly aggressive and oscillate more, while increasing k_d caused the vehicle to react too slowly to rapid geometric changes. We determined that the recommended default values $k_p = 6$ and $k_d = 4$ provided a balance between stability and agility, allowing the vehicle to navigate the hallway with minimal oscillation and execute turns safely.

LiDAR update rates are a critical constraint on gain selection. Low frequency data arrival introduces feedback delays. To prevent instability in such cases, k_p should be lowered to reduce the system's "aggressiveness". High-frequency scanning, however, enables more aggressive tuning and faster transient responses. Our testing confirmed that the provided default gains were optimized for the lab's specific LiDAR hardware, achieving a stable balance across all navigation scenarios. In general, tuning should always account for sensor latency to avoid a mismatch between the control logic and the physical state of the vehicle.

Activity 5:

1) While testing the vehicle behavior in real world, we first observed that the LiDAR orientation was shifted by 180 degrees from the `base_link` so our code needed to be adjusted accordingly. A deviation between the vehicle behaviour in simulations and the experiment is that our vehicle tends to have stronger oscillations but still centered between the walls. Upon observation, we also do realize that the vehicle tends to still steer towards the wall sometimes even in a straight hallway before quickly adjusting itself. This behaviour was justified through the irregularities in the hallways where doors and door frames are present. These reflective surfaces can result in some inconsistent readings where the LiDAR doesn't get accurate distance measurements or returns infinity. In our implementation, any scanned ranges larger than the max LiDAR range set are automatically replaced with the maximum allowed LiDAR distance of 12 meters. This can momentarily skew the vehicle's perception of the wall as it may force the program to believe the vehicle is 12 meters away at one point. Real-world testing revealed limitations that were not present in the simulation, specifically regarding sensor noise and wall irregularities. Obstacles near the path, such as boxes, were occasionally misinterpreted as the wall boundary, leading to brief steering errors. The system's continuous data acquisition allowed for rapid self-correction, but these disturbances highlight a need for improved signal processing. Implementing an averaging algorithm across multiple LiDAR points would reduce the impact of

these outliers. This would “smooth” the perceived wall geometry and mitigate the influence of small environmental features on the overall control law.

2) The relationship between mechanical scan frequency and sampling rate defines a critical trade-off between update latency and the angular resolution. If the scan frequency is increased while the sampling rate remains fixed, the angular density of the data decreases. For example, doubling the scan frequency effectively halves the points per revolution, potentially causing the system to overlook thin obstacles or smaller geometric features. Despite the reduction in spatial density, higher scan frequencies minimize transport lag, providing the controller with more frequent state updates. This is vital for high-speed navigations or dynamic environments requiring rapid reflex actions. Conversely, a lower scan frequency maximizes spatial resolution and point density, which is beneficial for fast mapping and precision control, but at the cost of increased feedback delay.

Activity 7:

There were several key parameters within the control algorithm that we could adjust to optimize the vehicle’s performance. The most critical factors identified were the nominal velocity, safe distance, stop distance, and the FOV angle. While we could have adjusted the proportional and derivative gains, extensive testing confirmed that the default values of $k_p = 6$ and $k_d = 4$ provided the best foundational stability for our hardware.

During initial simulations, a nominal velocity of 2.0 m/s appeared stable for both wall-following and cornering. However, real-world testing with the virtual barrier algorithm revealed that this speed was excessive. At 2.0 m/s, the vehicle’s momentum and minor steering oscillations made it difficult for the controller to correct its trajectory in time to navigate sharp corners or sudden obstacles safely. After conducting iterative tests, we selected the recommended 1.0 m/s as the optimal speed. This provided a necessary balance; the vehicle remained responsive enough to decelerate smoothly using the exponential decay parameter (d_r) while maintaining enough forward velocity to navigate the hallway efficiently. This speed reduction prioritized collision avoidance, ensuring that the system had sufficient time to react to the environment.

The d_{stop} parameter defines the “critical zone” where the vehicle must come to a complete halt. Initially set to 0.6 meters, we found this caused the vehicle to be overly cautious, often stopping prematurely. By reducing d_{stop} to 0.4 meters, we improved the vehicle’s ability to maneuver around obstacles smoothly. With a shorter stopping distance, the algorithm was able to compute more accurate steering commands closer to the obstacle, keeping the vehicle on its intended path rather than drifting away too early. The trade-off here is safety: while 0.4m allows for tighter maneuvering, it leaves less room for error if the sensors fail to detect an object.

The FOV angle determines the width of the vehicle’s vision on either side. We initially used the recommended 70-degree FOV. When we experimented with narrower ranges (20–30 degrees), the performance degraded significantly. The algorithm relies on detecting gaps in the LiDAR data to identify turns. If the FOV is too narrow, the vehicle cannot see the turn until it is already passing it, leading to missed corners. Therefore, we stayed on a 70-degree FOV. This provided enough situational awareness to detect side walls and upcoming turns without becoming overly

sensitive to minor hallway features like doorway recesses. While increasing the FOV further can improve awareness, it also increases the risk of the vehicle trying to turn into small dips in the wall.

The d_{safe} parameter defines the radius at which objects are officially flagged as obstacles. We set this to 2.0 meters, which allowed the vehicle to detect upcoming walls early enough to plan its steering. If d_{safe} is too low (e.g., 0.5m), the vehicle effectively becomes blind to the hallway geometry until a collision is approaching. However, a high d_{safe} value must be balanced with the FOV. If d_{safe} is too large in a narrow hallway, the entire FOV can become saturated with obstacles. When this happens, our algorithm's exception handling may fail to find a clear path, causing the vehicle to drive straight until a gap appears or the brakes engage. Our chosen 2.0m threshold was the maximum distance that provided early detection without triggering these saturated states in tight spaces.

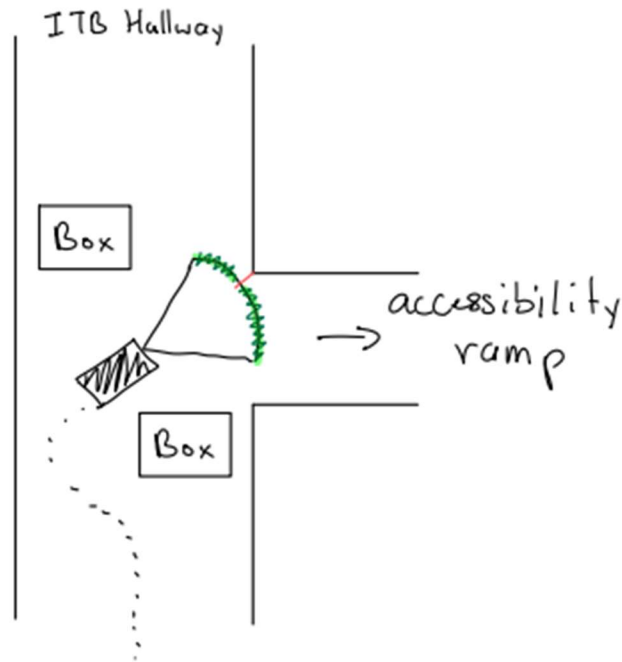
Activity 8:

When testing the virtual barrier following algorithm in the ITB hallway, the system performed as expected. To ensure a fair comparison with our simulation results, we used cardboard boxes as physical obstacles, positioning them in a layout similar to our virtual environment. In terms of obstacle avoidance, the vehicle's behavior in the real world mirrored the simulation, though it navigated around the boxes with slower maneuvering.

One of our primary goals was to maintain consistency between the simulation and real-life parameters. We initially reduced the stopping distance to 0.4 m in the simulation to provide the controller with a larger operational buffer when navigating near obstacles. During real-world testing, we further reduced this to 0.2 m. Because the physical vehicle exhibited less reactivity than the simulation, lowering the d_{stop} threshold was necessary to grant the steering algorithm more room to execute corrections before the safety-stop logic was triggered.

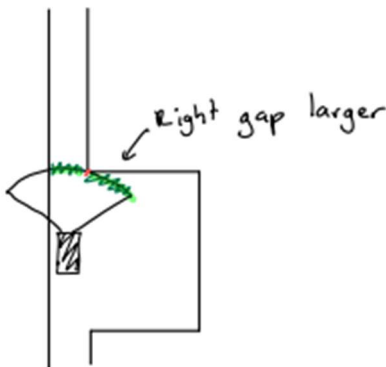
We also performed iterative tuning of the controller gains to better suit the physical environment. While the simulation initially suggested that the recommended values of $k_p = 6$ and $k_d = 4$ were sufficient, real-world testing revealed inconsistent performance, characterized by significant overshoots when navigating between obstacles. This causes the vehicle to not have enough turning space to navigate through the remaining obstacles. These overshoots indicated that the proportional response was too aggressive for the vehicle's physical steering dynamics. By reducing $k_p = 4$, we successfully dampened the system's sensitivity, resulting in smoother trajectory corrections and more reliable obstacle avoidance.

We also did notice a flaw when finding the best direction of travel. When testing in the real world environment, we had an obstacle placement next to an open section in the hallway as shown below:



Although this issue was our fault of environment setup, it was a case that caught our attention as this can occur if there is a larger gap or dead end on either side that appears as a “better” direction of travel.

In a case where there is a narrow hallway and a large dead end gap on the sides, the vehicle algorithm will navigate itself into the wrong path, as shown below:



Although we didn’t necessarily have this issue, we did think about these edge cases and how we would resolve them. One effective method is to have a more narrow FOV by decreasing the FOV angle, in turn would make the front gap appear “wider”, therefore being the best direction of travel. An alternative solution is to also increase the safe distance threshold, which can result in detecting the wall on the sides as an obstacle sooner.

Activity 9:

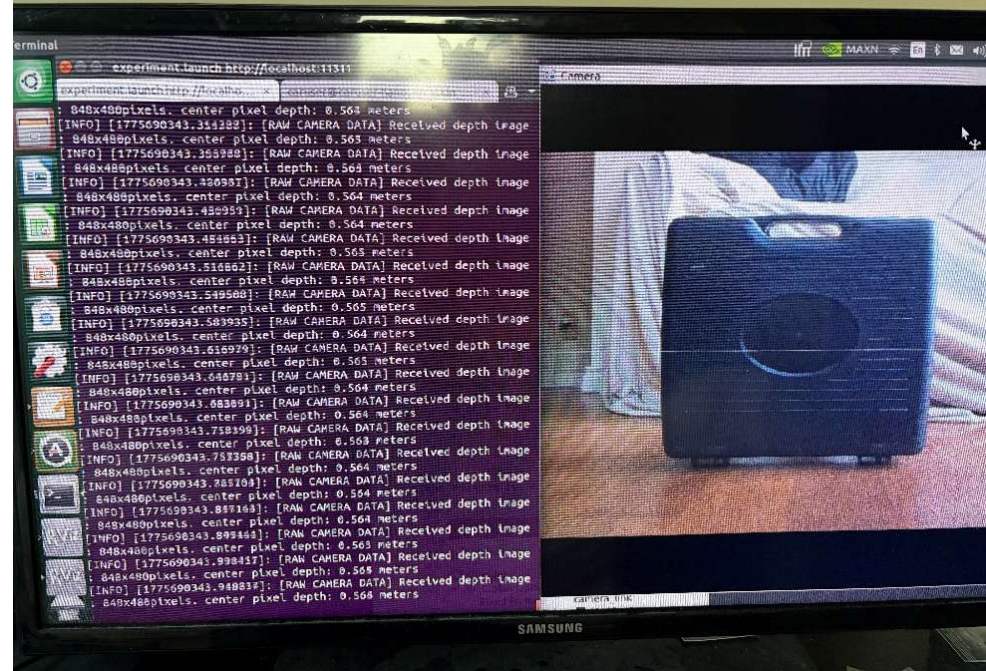
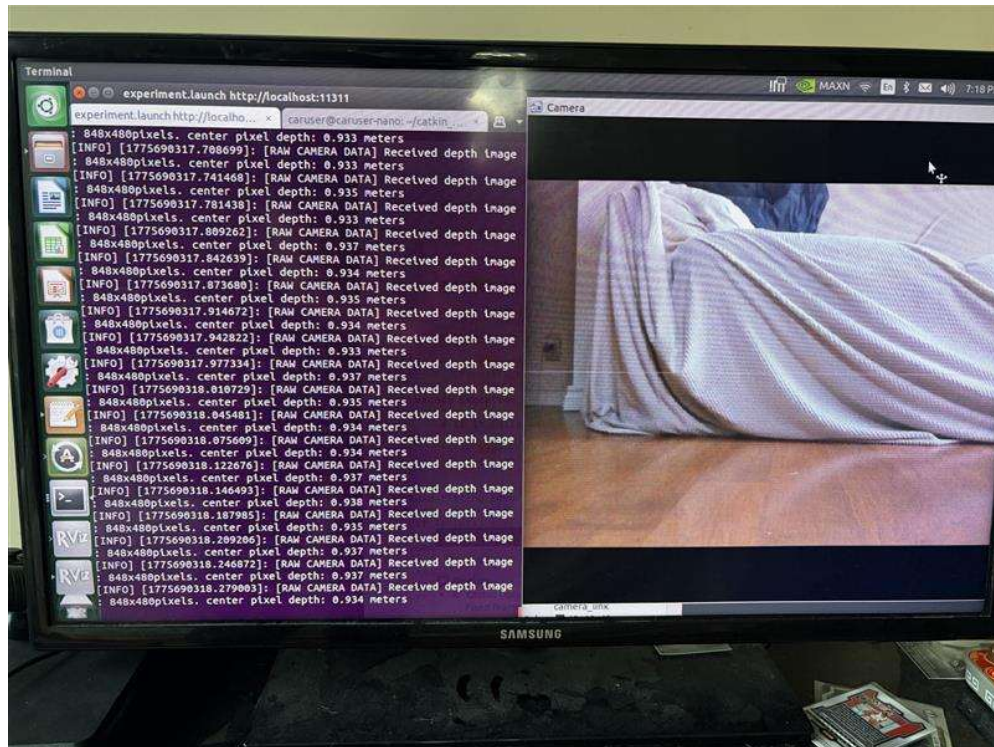
In this activity, we used the RGB-D camera depth image information for detecting and avoiding obstacles undetectable by the LiDAR. We extended our barrier navigation node to also accomplish the following:

- 1) Subscribes and listens to the Depth Image data and Depth Information topics published by the Realsense RGB-Depth camera.
- 2) Obtains the depth camera intrinsic parameters from the data published in the Depth camera information topic.

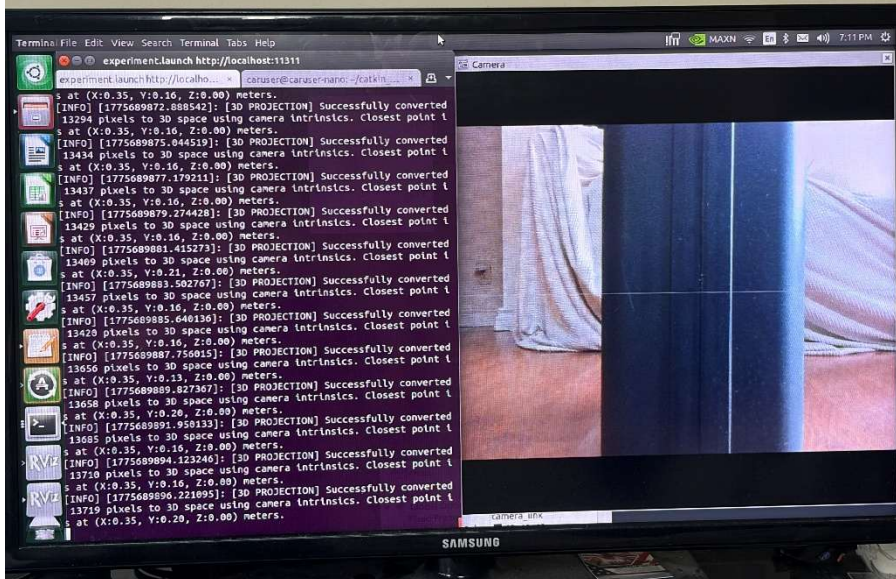
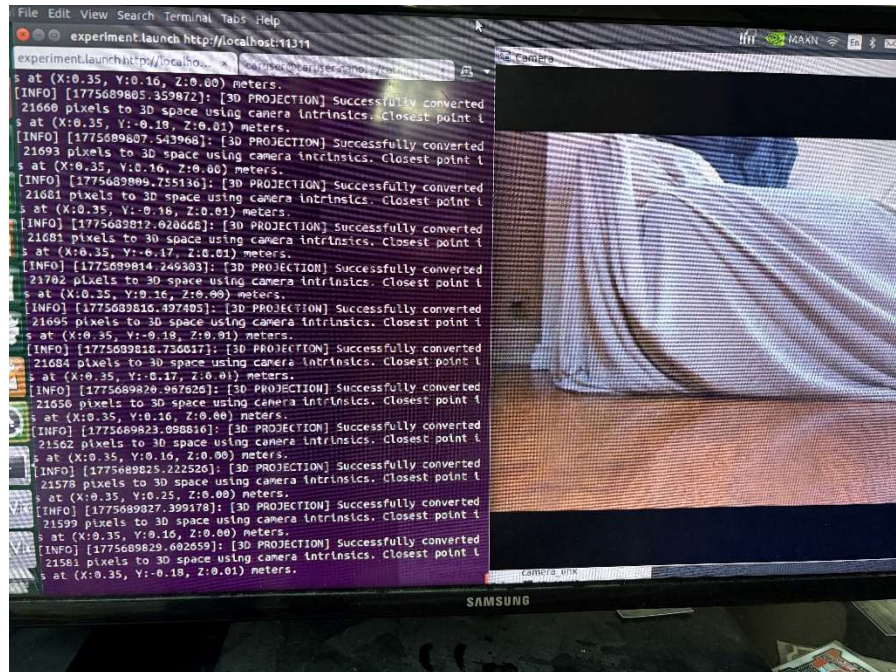
```
rev ids: accel:251 mag:50 gyro:15 sw:785 bl:21  
[vesc_driver_node-2] process has finished cleanly  
log file: /home/caruser/.ros/log/0c5af4a2-32be-11f1-b29d-a46bb60709c7/vesc_driver_node  
[ INFO] [1775594876.773630460]: Experiment constructed.  
[INFO] [1775594877.998644]: D435 intrinsics: fx=420.0 fy=420.0 cx=426.8 cy=241.1
```

- 3) Retrieves the depth image data from the Depth Image topic.

```
5593994.350544]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.218 meters  
5593994.384908]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.228 meters  
5593994.417613]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.360 meters  
5593994.456168]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.355 meters  
5593994.488139]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.358 meters  
5593994.519640]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.355 meters  
5593994.551770]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.360 meters  
5593994.586820]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.352 meters  
5593994.625217]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.347 meters  
5593994.652205]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.355 meters  
5593994.684799]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.360 meters  
5593994.718792]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 1.358 meters  
5593994.752910]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.340 meters  
5593994.789087]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.350 meters  
5593994.817732]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.359 meters  
5593994.850671]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.369 meters  
5593994.885685]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.379 meters  
5593994.921221]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.387 meters  
5593994.950885]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.393 meters  
5593994.983661]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.398 meters  
5593995.023535]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.403 meters  
5593995.054485]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.407 meters  
5593995.084492]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.407 meters  
5593995.118450]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.412 meters  
5593995.150785]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.410 meters  
5593995.184031]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.414 meters  
5593995.218037]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.405 meters  
5593995.250202]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.393 meters  
5593995.282585]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.382 meters  
5593995.354251]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.371 meters  
5593995.386108]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.360 meters  
5593995.423583]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.353 meters  
5593995.456935]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.348 meters  
5593995.484220]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.345 meters  
5593995.510155]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.352 meters  
5593995.552518]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.359 meters  
5593995.584715]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.366 meters  
5593995.619758]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.374 meters  
5593995.652918]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.388 meters  
5593995.685553]: [RAW CAMERA DATA] Received depth image: 848x480pxels. center pixel depth: 0.395 meters
```

- 4) Converts the depth image pixels to the points in 3D space using the camera model and camera intrinsic parameters.



- 5) Finds obstacle points that can potentially collide with the vehicle but are undetectable by LiDAR and overwrites the LiDAR decision making with what the camera sees.

However, our implementation could not be fully concluded as we were unable to properly validate the sensor fusion algorithm in a unified environment. While we successfully initialized the depth camera nodes using the command `roslaunch realsense2_camera rs_camera.launch` and visualized the real-time feed via `roslaunch rviz rviz`, our testing relied on a hybrid setup passing real-world depth data into the `simulator.launch` environment. Because of this environmental disconnect, we were unable to accurately test the algorithm responsible for isolating points that could potentially collide with the vehicle. Our experimental setup resulted in continuous false positives, where the camera always perceived an obstacle in its field of view. It is also uncertain that our algorithm is failing to filter out the ground plane which may be causing the camera to always report a massive obstacle. Consequently, the vehicle was forced into a state where it always registered an approaching collision, preventing us from definitively concluding that the camera and LiDAR algorithms would seamlessly cooperate in a shared, real-world environment.